

AI Customer Support Agent — AmazonHelp

Hiver SDE Intern Take-Home Assignment

Executive summary. This project builds a lightweight customer-support agent for AmazonHelp using a prepared subsample of the Customer Support on Twitter (TWCS) dataset. The system combines intent classification, retrieval of historical support responses, and a conservative auto-handle/escalation decision. Evaluation uses 198 labeled examples with a 50-example held-out test split.

1. Problem framing

The goal is to route customer-support requests to a useful intent and either provide a grounded response based on historical support interactions or escalate uncertain/high-risk cases for review. AmazonHelp was selected as the target brand. The implementation is deliberately lightweight and reproducible rather than dependent on a large generative model.

2. Data and preprocessing

The source is the public Customer Support on Twitter (TWCS) dataset. A prepared AmazonHelp subset is included with the project so the experiment can be run without downloading the full multi-million-row dataset. Customer tweets are paired with their response tweets, and the resulting text is normalized for modeling and retrieval. The gold set contains 198 examples; labels are AI-assisted seed labels with targeted review rather than independently double-annotated human labels.

3. Intent taxonomy and agent design

The taxonomy contains 13 intents: `account_access`, `billing_payment`, `subscription_refund`, `order_delivery`, `order_purchase`, `device_setup_activation`, `app_or_service_failure`, `connectivity`, `media_content`, `feature_howto`, `security_privacy`, `device_hardware`, `other_escalate`.

The classifier uses TF-IDF features with Logistic Regression. The support agent also retrieves a similar historical customer message and response. A conservative decision layer uses model confidence/retrieval evidence and escalates uncertain cases instead of automatically answering every request.

4. Baselines and evaluation

System	Accuracy	Macro-F1
Majority baseline	8.00%	1.14%
TF-IDF + Logistic Regression	44.00%	40.59%

The classifier improves substantially over the majority baseline, but 44% accuracy should not be interpreted as production performance. The held-out test set is only 50 examples and several intents have very low support, making per-class estimates unstable.

5. Agent smoke test

Input: "Where is my Amazon package?"

Predicted intent: **order_delivery**

Retrieval similarity: **1.0**

Decision: **auto_handle**

A historical AmazonHelp response was retrieved successfully.

6. Failure analysis

- Order/delivery language can be highly repetitive, so lexical similarity can produce very high retrieval scores even when the exact customer context differs.
- Intent boundaries such as account access vs. app/service failure can overlap when a customer reports both inability to sign in and an application problem.
- Connectivity and application failures can share terms such as "not working," "connection," and "network," creating classification ambiguity.
- Security/privacy requests require conservative handling because an apparently ordinary support question may involve sensitive account or payment information.
- Low-support intents are difficult to evaluate reliably; a single held-out example can materially change a class-level score.

7. What is misleading about the headline number?

The 44.00% accuracy is useful for comparing this baseline with the majority baseline, but it is not a production-quality estimate. The test set contains only 50 examples and the class distribution is uneven. Macro-F1 is therefore also sensitive to low-support classes. A stronger evaluation would use a larger, stratified, independently human-reviewed test set and confidence intervals.

8. Non-obvious design decisions

1. Use a brand-specific subset rather than mixing all support brands.
2. Pair inbound customer tweets with their response tweets.
3. Keep a compact intent taxonomy instead of dozens of fine-grained labels.
4. Use TF-IDF + Logistic Regression as an interpretable baseline.
5. Use retrieval to ground replies in historical support language.
6. Separate intent confidence from retrieval similarity.
7. Prefer escalation when evidence is weak.
8. Keep security/privacy handling conservative.
9. Use a held-out split rather than evaluating on training examples.
10. Report both accuracy and macro-F1 because class imbalance matters.
11. Retain the original customer/response pair for auditability.
12. Treat low-support class metrics as unstable.
13. Keep raw TWCS data out of the repository because of size/licensing considerations.
14. Make the experiment runnable from prepared project data.
15. Expose the agent as a simple command-line smoke test for reproducibility.

9. Limitations and next steps

Next steps would include expanding the independently human-reviewed gold set, stratifying evaluation across intents, testing stronger transformer-based classifiers, adding calibrated confidence estimates, improving retrieval with semantic embeddings, adding PII/safety filters, and evaluating response quality with human or LLM-assisted judging. The current results should be treated as a take-home prototype rather than a production deployment.

10. Reproducibility

The repository contains the prepared data, source code, tests, documentation, and evaluation artifacts. The main commands are: `python -m src.run_experiment --gold data/gold_set.csv`, `python -m src.agent --message "Where is my Amazon package?"`, and `pytest -q` with `PYTHONPATH=.` on Windows PowerShell.